Créer une application de chat type Messenger : messages texte, images, fichiers et audio

Guide complet
WebSocket + React.js + NestJS +PostgreSQL

Formateur : Salem Boubaker

1. Vue d'ensemble

Une application de chat moderne mélange généralement plusieurs briques : une interface React pour afficher les conversations, un backend NestJS pour gérer la logique métier, une base PostgreSQL pour conserver les données, et un canal WebSocket pour la communication temps réel. L'idée est simple : quand un utilisateur envoie un message, le client notifie le serveur via WebSocket, le serveur enregistre ce message en base, puis diffuse immédiatement l'événement aux autres utilisateurs connectés à la même conversation.

- React.js : affichage des discussions, saisie du message, aperçu des pièces jointes, lecture audio.
- NestJS : gestion des rooms, validation des données, envoi temps réel via gateway WebSocket.
- PostgreSQL : stockage des utilisateurs, des conversations, des participants et des messages.
- HTTP : upload des images, des fichiers et des notes vocales.
- WebSocket : notification instantanée des nouveaux messages, présence, lecture, frappe en cours.

Pourquoi séparer HTTP et WebSocket ? Le binaire lourd (image, audio, document) passe mieux par HTTP. Le WebSocket sert surtout à signaler qu'un nouveau message existe et à synchroniser tous les clients.

2. Architecture cible

Pour rester proche d'une application de type Messenger, nous allons organiser le projet autour de quatre flux principaux : authentification minimale, consultation des conversations, envoi des messages, et synchronisation en temps réel.

Schéma logique

```
[React Client]
  |-- HTTP --> /auth, /rooms, /messages, /upload
  |-- WS   --> /chat (join room, new message, typing, seen)

[NestJS Backend]
  |-- REST Controllers
  |-- WebSocket Gateway
  |-- Services métier
  |-- Prisma / TypeORM
  |-- PostgreSQL

[Stockage fichier]
  |-- dossier /uploads ou stockage cloud
```

Dans cette version pédagogique, chaque conversation est une room Socket.IO. Quand un client ouvre une conversation, il rejoint cette room. Le serveur diffuse ensuite tous les nouveaux messages vers cette room uniquement. Ainsi, seuls les participants concernés reçoivent les événements.

3. Modèle de données PostgreSQL

Avant d'écrire le moindre écran, il faut définir clairement les données à stocker. Pour un chat simple mais robuste, quatre tables suffisent au départ : users, rooms, room_participants et messages.

SQL de base

```
-- Table des utilisateurs.
CREATE TABLE users (
    id UUID PRIMARY KEY,
    full_name VARCHAR(120) NOT NULL,
    email VARCHAR(160) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    avatar_url TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Table des conversations.
CREATE TABLE rooms (
    id UUID PRIMARY KEY,
    name VARCHAR(160),
    is_group BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Table de liaison entre room et utilisateur.
CREATE TABLE room_participants (
    room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    joined_at TIMESTAMP NOT NULL DEFAULT NOW(),
    PRIMARY KEY (room_id, user_id)
);

-- Table des messages.
CREATE TABLE messages (
    id UUID PRIMARY KEY,
    room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
    sender_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    type VARCHAR(20) NOT NULL, -- text, image, file, audio
    content TEXT,
    file_url TEXT,
    file_name VARCHAR(255),
    mime_type VARCHAR(120),
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    seen_at TIMESTAMP
);
```

Le champ type permet de savoir comment afficher le message côté frontend. Par exemple, un message de type text utilisera le champ content, alors qu'un message de type image utilisera surtout file_url.

4. Préparation du backend NestJS

1. Créer le projet NestJS.

Installation

```
# 1) Installer le CLI NestJS si nécessaire.
npm install -g @nestjs/cli

# 2) Créer le projet backend.
nest new chat-backend

# 3) Entrer dans le dossier.
cd chat-backend

# 4) Installer les dépendances nécessaires.
npm install @nestjs/websockets @nestjs/platform-socket.io
npm install @nestjs/config @nestjs/serve-static
npm install @nestjs/typeorm typeorm pg
npm install class-validator class-transformer
npm install uuid
npm install @nestjs/platform-express multer
```

Nous utilisons ici TypeORM parce qu'il s'intègre bien avec NestJS pour un guide pas à pas. PostgreSQL sera le moteur relationnel principal.

2. Configurer les variables d'environnement.

Fichier .env

```
# Fichier .env
PORT=3000
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_USER=postgres
DATABASE_PASSWORD=postgres
DATABASE_NAME=chat_app
CLIENT_URL=http://localhost:5173
```

3. Configurer le module principal et la connexion PostgreSQL.

Configuration principale

```
// src/app.module.ts
import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { ServeStaticModule } from '@nestjs/serve-static';
import { join } from 'path';
import { ChatModule } from './chat/chat.module';
import { User } from './chat/entities/user.entity';
import { Room } from './chat/entities/room.entity';
import { RoomParticipant } from './chat/entities/room-participant.entity';
import { Message } from './chat/entities/message.entity';

@Module({
  imports: [
    // Charge les variables du fichier .env.
    ConfigModule.forRoot({ isGlobal: true }),

    // Expose le dossier uploads pour servir les images et fichiers.
    ServeStaticModule.forRoot({
      rootPath: join(__dirname, '..', 'uploads'),
      serveRoot: '/uploads',
    }),

    // Configure TypeORM de manière asynchrone.
    TypeOrmModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        type: 'postgres',
        host: config.get<string>('DATABASE_HOST'),
        port: Number(config.get<string>('DATABASE_PORT')),
        username: config.get<string>('DATABASE_USER'),
        password: config.get<string>('DATABASE_PASSWORD'),
        database: config.get<string>('DATABASE_NAME'),
        entities: [User, Room, RoomParticipant, Message],
        synchronize: true,
      }),
    }),

    // Module métier du chat.
    ChatModule,
  ],
})
export class AppModule {}
```

Le paramètre synchronize: true est pratique en développement. En production, il faut plutôt utiliser des migrations pour mieux contrôler l’évolution de la base.

5. Entités TypeORM

Les entités décrivent les tables côté TypeScript. Elles servent aussi à clarifier les relations entre les objets du domaine.

Exemple : entité User

```
// src/chat/entities/user.entity.ts
import { Column, Entity, OneToMany, PrimaryGeneratedColumn } from 'typeorm';
import { Message } from './message.entity';
import { RoomParticipant } from './room-participant.entity';

@Entity('users')
export class User {
  // Génère automatiquement un identifiant UUID.
  @PrimaryGeneratedColumn('uuid')
  id: string;

  // Nom complet affiché dans l'interface.
  @Column({ name: 'full_name', length: 120 })
  fullName: string;

  // Adresse email unique.
  @Column({ unique: true, length: 160 })
  email: string;

  // Mot de passe haché.
  @Column({ name: 'password_hash', type: 'text' })
  passwordHash: string;

  // Avatar éventuel.
  @Column({ name: 'avatar_url', type: 'text', nullable: true })
  avatarUrl?: string;

  // Date de création de l'utilisateur.
  @Column({ name: 'created_at', type: 'timestamp', default: () => 'NOW()' })
  createdAt: Date;

  // Liste des messages envoyés par cet utilisateur.
  @OneToMany(() => Message, (message) => message.sender)
  messages: Message[];

  // Liste des participations aux rooms.
  @OneToMany(() => RoomParticipant, (participant) => participant.user)
  roomParticipations: RoomParticipant[];
}
```

Exemple : entité Message

```
// src/chat/entities/message.entity.ts
import { Column, Entity, JoinColumn, ManyToOne, PrimaryGeneratedColumn } from 'typeorm';
import { Room } from './room.entity';
import { User } from './user.entity';

@Entity('messages')
export class Message {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ name: 'room_id' })
  roomId: string;

  @Column({ name: 'sender_id' })
  senderId: string;

  // Type du message : text, image, file ou audio.
  @Column({ length: 20 })
  type: string;

  // Contenu texte du message.
  @Column({ type: 'text', nullable: true })
  content?: string;

  // URL du fichier stocké sur le serveur.
  @Column({ name: 'file_url', type: 'text', nullable: true })
  fileUrl?: string;

  @Column({ name: 'file_name', length: 255, nullable: true })
  fileName?: string;

  @Column({ name: 'mime_type', length: 120, nullable: true })
  mimeType?: string;

  @Column({ name: 'created_at', type: 'timestamp', default: () => 'NOW()' })
  createdAt: Date;

  @Column({ name: 'seen_at', type: 'timestamp', nullable: true })
  seenAt?: Date;

  @ManyToOne(() => Room, (room) => room.messages, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'room_id' })
  room: Room;

  @ManyToOne(() => User, (user) => user.messages, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'sender_id' })
  sender: User;
}
```

Les fichiers room.entity.ts et room-participant.entity.ts suivent la même logique. L'important est de bien représenter les relations : une room contient plusieurs messages, et plusieurs utilisateurs peuvent appartenir à une room.

6. DTO et validation

Dans NestJS, un DTO sert à valider les données reçues. C'est essentiel pour éviter d'accepter des messages incomplets ou mal formés.

DTO de création de message

```
// src/chat/dto/create-message.dto.ts
import { IsIn, IsOptional, IsString, IsUUID } from 'class-validator';

export class CreateMessageDto {
  // Identifiant de la room cible.
  @IsUUID()
  roomId: string;

  // Identifiant de l'utilisateur qui envoie le message.
  @IsUUID()
  senderId: string;

  // Type du message.
  @IsString()
  @IsIn(['text', 'image', 'file', 'audio'])
  type: 'text' | 'image' | 'file' | 'audio';

  // Texte éventuel.
  @IsOptional()
  @IsString()
  content?: string;

  // URL du fichier si ce n'est pas un simple texte.
  @IsOptional()
  @IsString()
  fileUrl?: string;

  @IsOptional()
  @IsString()
  fileName?: string;

  @IsOptional()
  @IsString()
  mimeType?: string;
}
```

7. Service métier du chat

Le service centralise la logique métier : créer un message, vérifier que l'utilisateur appartient à la room, l'enregistrer en base, puis renvoyer l'objet final prêt à diffuser.

Service principal

```
// src/chat/chat.service.ts
import { BadRequestException, Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Message } from './entities/message.entity';
import { RoomParticipant } from './entities/room-participant.entity';
import { CreateMessageDto } from './dto/create-message.dto';

@Injectable()
export class ChatService {
  constructor(
    @InjectRepository(Message)
    private readonly messageRepository: Repository<Message>,

    @InjectRepository(RoomParticipant)
    private readonly participantRepository: Repository<RoomParticipant>,
  ) {}

  async createMessage(dto: CreateMessageDto) {
    // Vérifie que l'utilisateur appartient bien à la room.
    const participant = await this.participantRepository.findOne({
      where: { roomId: dto.roomId, userId: dto.senderId },
    });

    if (!participant) {
      throw new NotFoundException('Utilisateur non autorisé dans cette room');
    }

    // Refuse un message texte vide.
    if (dto.type === 'text' && !dto.content?.trim()) {
      throw new BadRequestException('Le message texte est vide');
    }

    // Crée l'entité à partir du DTO.
    const message = this.messageRepository.create({
      roomId: dto.roomId,
      senderId: dto.senderId,
      type: dto.type,
      content: dto.content,
```

```
      fileUrl: dto.fileUrl,
      fileName: dto.fileName,
      mimeType: dto.mimeType,
    });

    // Sauvegarde en base de données.
    const savedMessage = await this.messageRepository.save(message);

    // Retourne le message final pour diffusion.
    return savedMessage;
  }

  async findMessagesByRoom(roomId: string) {
    return this.messageRepository.find({
      where: { roomId },
      order: { createdAt: 'ASC' },
    });
  }
}
```

8. WebSocket Gateway

Le gateway est le cœur temps réel de l'application. Il reçoit les événements envoyés par le client React et diffuse les mises à jour aux autres participants.

Gateway WebSocket

```
// src/chat/chat.gateway.ts
import {
  ConnectedSocket,
  MessageBody,
  OnGatewayConnection,
  OnGatewayDisconnect,
  SubscribeMessage,
  WebSocketGateway,
  WebSocketServer,
} from '@nestjs/websockets';
import { Server, Socket } from 'socket.io';
import { ChatService } from './chat.service';
import { CreateMessageDto } from './dto/create-message.dto';

@WebSocketGateway({
  cors: {
    origin: 'http://localhost:5173',
    credentials: true,
  },
```

```
})
export class ChatGateway implements OnGatewayConnection, OnGatewayDisconnect {
  // Instance Socket.IO du serveur.
  @WebSocketServer()
  server: Server;

  constructor(private readonly chatService: ChatService) {}

  handleConnection(client: Socket) {
    console.log(`Client connecté : ${client.id}`);
  }

  handleDisconnect(client: Socket) {
    console.log(`Client déconnecté : ${client.id}`);
  }

  @SubscribeMessage('join_room')
  async handleJoinRoom(
    @MessageBody() data: { roomId: string },
    @ConnectedSocket() client: Socket,
  ) {
    // Le client rejoint la room Socket.IO.
    await client.join(data.roomId);

    // Informe le client que l'opération a réussi.
    return { event: 'joined_room', data: { roomId: data.roomId } };
  }

  @SubscribeMessage('send_message')
  async handleSendMessage(@MessageBody() dto: CreateMessageDto) {
    // Crée le message et l'enregistre en base.
    const savedMessage = await this.chatService.createMessage(dto);

    // Diffuse le nouveau message à tous les clients dans la room.
    this.server.to(dto.roomId).emit('new_message', savedMessage);

    // Retourne aussi un accusé de réception au client émetteur.
    return { event: 'message_saved', data: savedMessage };
  }

  @SubscribeMessage('typing')
  handleTyping(
    @MessageBody() data: { roomId: string; userId: string; isTyping: boolean },
  ) {
    // Informe les autres membres qu'un utilisateur est en train d'écrire.
    this.server.to(data.roomId).emit('user_typing', data);

    return { ok: true };
```

```
  }

  @SubscribeMessage('seen_messages')
  handleSeen(@MessageBody() data: { roomId: string; userId: string }) {
    // Exemple simple de notification de lecture.
    this.server.to(data.roomId).emit('messages_seen', data);

    return { ok: true };
  }
}
```

9. Contrôleurs REST : rooms, messages et upload

Tout ne doit pas passer par WebSocket. L'historique initial d'une conversation et l'upload des fichiers sont plus simples en HTTP.

REST + upload

```
// src/chat/chat.controller.ts
import {
  Body,
  Controller,
  Get,
  Param,
  Post,
  UploadedFile,
  UseInterceptors,
} from '@nestjs/common';
import { FileInterceptor } from '@nestjs/platform-express';
import { diskStorage } from 'multer';
import { extname } from 'path';
import { ChatService } from './chat.service';
import { CreateMessageDto } from './dto/create-message.dto';

@Controller('chat')
export class ChatController {
  constructor(private readonly chatService: ChatService) {}

  @Get('rooms/:roomId/messages')
  async getMessages(@Param('roomId') roomId: string) {
    // Retourne tous les messages d'une room.
    return this.chatService.findMessagesByRoom(roomId);
```

```
  }

  @Post('messages')
  async createMessage(@Body() dto: CreateMessageDto) {
    // Variante REST si on veut créer un message sans socket.
    return this.chatService.createMessage(dto);
  }

  @Post('upload')
  @UseInterceptors(
    FileInterceptor('file', {
      storage: diskStorage({
        destination: './uploads',
        filename: (_req, file, callback) => {
          const uniqueName = `${Date.now()}-${Math.round(Math.random() * 1e9)}${extname(file.originalname)}`;
          callback(null, uniqueName);
        },
      }),
    }),
  )
  uploadFile(@UploadedFile() file: Express.Multer.File) {
    // Renvoie les métadonnées du fichier pour créer ensuite un message.
    return {
      fileUrl: `/uploads/${file.filename}`,
      fileName: file.originalname,
      mimeType: file.mimetype,
      size: file.size,
    };
  }
}
```

10. Module chat

Module métier

```
// src/chat/chat.module.ts
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { ChatController } from './chat.controller';
import { ChatGateway } from './chat.gateway';
import { ChatService } from './chat.service';
import { Message } from './entities/message.entity';
import { Room } from './entities/room.entity';
import { RoomParticipant } from './entities/room-participant.entity';
import { User } from './entities/user.entity';
```

```
@Module({
  imports: [TypeOrmModule.forFeature([User, Room, RoomParticipant, Message])],
  controllers: [ChatController],
  providers: [ChatService, ChatGateway],
})
export class ChatModule {}
```

11. Bootstrap de l’application NestJS

main.ts

```
// src/main.ts
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  // Crée l’application NestJS.
  const app = await NestFactory.create(AppModule);

  // Active la validation automatique des DTO.
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      transform: true,
      forbidNonWhitelisted: true,
    }),
  );

  // Autorise le frontend React.
  app.enableCors({
    origin: 'http://localhost:5173',
    credentials: true,
  });

  // Lance le serveur HTTP.
  await app.listen(3000);
}

bootstrap();
```

12. Préparation du frontend React

4. Créer le projet React avec Vite.

Installation React

```
# Créer le frontend.
npm create vite@latest chat-frontend -- --template react-ts

# Entrer dans le dossier.
cd chat-frontend

# Installer les dépendances.
npm install
npm install socket.io-client axios
```

5. Organiser l'arborescence React.

Arborescence recommandée

```
src/
  api/
    http.ts
  components/
    ChatLayout.tsx
    ConversationList.tsx
    MessageList.tsx
    MessageInput.tsx
    UploadPreview.tsx
  hooks/
    useChatSocket.ts
  pages/
    ChatPage.tsx
  types/
    chat.ts
  App.tsx
  main.tsx
```

13. Typages du frontend

Types TypeScript

```
// src/types/chat.ts
export type MessageType = 'text' | 'image' | 'file' | 'audio';

export interface ChatMessage {
  id: string;
  roomId: string;
  senderId: string;
```

```
  type: MessageType;
  content?: string;
  fileUrl?: string;
  fileName?: string;
  mimeType?: string;
  createdAt: string;
  seenAt?: string;
}

export interface UploadResponse {
  fileUrl: string;
  fileName: string;
  mimeType: string;
  size: number;
}
```

14. Client HTTP React

Axios

```
// src/api/http.ts
import axios from 'axios';

// Instance Axios centralisée pour toutes les requêtes HTTP.
export const http = axios.create({
  baseURL: 'http://localhost:3000',
  withCredentials: true,
});
```

15. Hook de connexion WebSocket

Le hook useChatSocket centralise la connexion Socket.IO. Ainsi, les composants restent propres et réutilisables.

Hook socket

```
// src/hooks/useChatSocket.ts
import { useEffect, useMemo } from 'react';
import { io, Socket } from 'socket.io-client';

export function useChatSocket(): Socket {
  // Crée une seule instance de socket pour ce hook.
  const socket = useMemo(() => {
    return io('http://localhost:3000', {
```

```
      transports: ['websocket'],
      withCredentials: true,
    });
  }, []);

  useEffect(() => {
    // Journalise la connexion pour le débogage.
    socket.on('connect', () => {
      console.log('Socket connecté :', socket.id);
    });

    // Journalise la déconnexion.
    socket.on('disconnect', () => {
      console.log('Socket déconnecté');
    });

    // Nettoie la connexion quand le composant est démonté.
    return () => {
      socket.disconnect();
    };
  }, [socket]);

  return socket;
}
```

16. Page principale de chat

La page principale charge l'historique, rejoint la room en WebSocket, puis écoute les nouveaux messages.

Page principale

```
// src/pages/ChatPage.tsx
import { useEffect, useState } from 'react';
import { http } from '../api/http';
import { useChatSocket } from '../hooks/useChatSocket';
import { ChatMessage } from '../types/chat';
import { MessageList } from '../components/MessageList';
import { MessageInput } from '../components/MessageInput';

const ROOM_ID = '11111111-1111-1111-1111-111111111111';
const USER_ID = '22222222-2222-2222-2222-222222222222';

export function ChatPage() {
  // Liste des messages affichés dans la conversation.
  const [messages, setMessages] = useState<ChatMessage[]>([]);
```

```
  // Instance socket centralisée.
  const socket = useChatSocket();

  useEffect(() => {
    // Charge l'historique initial de la room.
    async function loadMessages() {
      const response = await http.get<ChatMessage[]>(`/chat/rooms/${ROOM_ID}/messages`);
      setMessages(response.data);
    }

    loadMessages();
  }, []);

  useEffect(() => {
    // Rejoint la room quand la socket est prête.
    socket.emit('join_room', { roomId: ROOM_ID });

    // Réagit à chaque nouveau message reçu en temps réel.
    const onNewMessage = (message: ChatMessage) => {
      setMessages((current) => [...current, message]);
    };

    socket.on('new_message', onNewMessage);

    // Nettoie l'écouteur quand le composant change ou se démonte.
    return () => {
      socket.off('new_message', onNewMessage);
    };
  }, [socket]);

  return (
    <div style={{ display: 'grid', gridTemplateRows: '1fr auto', height: '100vh' }}>
      <MessageList messages={messages} currentUserId={USER_ID} />
      <MessageInput socket={socket} roomId={ROOM_ID} senderId={USER_ID} />
    </div>
  );
}
```

17. Affichage des messages

Composant MessageList

```
// src/components/MessageList.tsx
import { ChatMessage } from '../types/chat';
```

```
interface Props {
  messages: ChatMessage[];
  currentUserId: string;
}

export function MessageList({ messages, currentUserId }: Props) {
  return (
    <div style={{ padding: 16, overflowY: 'auto', background: '#f5f7fb' }}>
      {messages.map((message) => {
        const isMine = message.senderId === currentUserId;

        return (
          <div
            key={message.id}
            style={{
              display: 'flex',
              justifyContent: isMine ? 'flex-end' : 'flex-start',
              marginBottom: 12,
            }}
          >
            <div
              style={{
                maxWidth: 420,
                padding: 12,
                borderRadius: 16,
                background: isMine ? '#2563eb' : '#ffffff',
                color: isMine ? '#ffffff' : '#111827',
                boxShadow: '0 1px 3px rgba(0,0,0,0.08)',
              }}
            >
              {message.type === 'text' && <p style={{ margin: 0 }}>{message.content}</p>}

              {message.type === 'image' && (
                <img
                  src={`http://localhost:3000${message.fileUrl}`}
                  alt={message.fileName || 'image'}
                  style={{ maxWidth: '100%', borderRadius: 12 }}
                />
              )}

              {message.type === 'audio' && (
                <audio controls style={{ width: '100%' }}>
                  <source src={`http://localhost:3000${message.fileUrl}`} type={message.mimeType} />
                </audio>
              )}

              {message.type === 'file' && (
                <a
```

```
              href={`http://localhost:3000${message.fileUrl}`}
              target="_blank"
              rel="noreferrer"
              style={{ color: isMine ? '#ffffff' : '#2563eb', fontWeight: 600 }}
            >
              {message.fileName}
            </a>
          )}
        </div>
      </div>
    );
  })}
  </div>
 );
}
```

18. Saisie et envoi d’un message texte

Composant MessageInput

```
// src/components/MessageInput.tsx
import { ChangeEvent, useState } from 'react';
import { Socket } from 'socket.io-client';
import { http } from '../api/http';
import { UploadResponse } from '../types/chat';

interface Props {
  socket: Socket;
  roomId: string;
  senderId: string;
}

export function MessageInput({ socket, roomId, senderId }: Props) {
  // Valeur du champ texte.
  const [text, setText] = useState('');

  // Fichier éventuellement sélectionné.
  const [selectedFile, setSelectedFile] = useState<File | null>(null);

  async function sendTextMessage() {
    // Ignore les messages vides.
    if (!text.trim()) return;

    // Envoie le message au serveur via WebSocket.
    socket.emit('send_message', {
      roomId,
```

```
    senderId,
    type: 'text',
    content: text,
  });

  // Vide le champ après envoi.
  setText('');
}

function handleFileChange(event: ChangeEvent<HTMLInputElement>) {
  // Récupère le premier fichier choisi.
  const file = event.target.files?.[0] ?? null;
  setSelectedFile(file);
}

async function uploadAndSendFile() {
  if (!selectedFile) return;

  // Prépare un FormData pour l'upload HTTP.
  const formData = new FormData();
  formData.append('file', selectedFile);

  // Envoie le binaire via HTTP.
  const response = await http.post<UploadResponse>('/chat/upload', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  });

  // Détermine le type logique du message.
  const mime = response.data.mimeType;
  const messageType = mime.startsWith('image/')
    ? 'image'
    : mime.startsWith('audio/')
    ? 'audio'
    : 'file';

  // Notifie ensuite le serveur via WebSocket.
  socket.emit('send_message', {
    roomId,
    senderId,
    type: messageType,
    fileUrl: response.data.fileUrl,
    fileName: response.data.fileName,
    mimeType: response.data.mimeType,
    content: text || undefined,
  });

  setSelectedFile(null);
  setText('');
```

```
  }

  return (
    <div style={{ display: 'flex', gap: 12, padding: 16, borderTop: '1px solid #e5e7eb' }}>
      <input
        type="text"
        value={text}
        onChange={(event) => setText(event.target.value)}
        onFocus={() => socket.emit('typing', { roomId, userId: senderId, isTyping: true })}
        onBlur={() => socket.emit('typing', { roomId, userId: senderId, isTyping: false })}
        placeholder="Écrire un message"
        style={{ flex: 1, padding: 12, borderRadius: 12, border: '1px solid #d1d5db' }}
      />

      <input type="file" onChange={handleFileChange} />

      <button onClick={sendTextMessage}>Envoyer texte</button>
      <button onClick={uploadAndSendFile}>Envoyer fichier</button>
    </div>
  );
}
```

19. Flux complet d'un message

Voici maintenant le chemin complet suivi par un message dans l'application. Cette partie est essentielle pour comprendre comment toutes les couches travaillent ensemble.

6. L'utilisateur saisit un message texte ou choisit un fichier dans l'interface React.
7. Si c'est un fichier, React l'envoie d'abord au backend via POST /chat/upload.
8. Le backend stocke le fichier dans le dossier uploads puis renvoie fileUrl, fileName et mimeType.
9. React déclenche ensuite socket.emit("send_message", payload).
10. Le gateway NestJS reçoit l'événement, appelle ChatService.createMessage(dto), puis enregistre le message en base.
11. Le gateway diffuse server.to(roomId).emit("new_message", savedMessage).
12. Tous les clients connectés à cette room reçoivent le message et mettent immédiatement l'interface à jour.

Résultat : Le même principe fonctionne pour le texte, l'image, le document et l'audio. La différence principale vient du type de message et de la façon dont le frontend l'affiche.

20. Exemple de fichier room.entity.ts

Entité Room

```
// src/chat/entities/room.entity.ts
import { Column, Entity, OneToMany, PrimaryGeneratedColumn } from 'typeorm';
import { Message } from './message.entity';
import { RoomParticipant } from './room-participant.entity';

@Entity('rooms')
export class Room {
  // Identifiant unique de la conversation.
  @PrimaryGeneratedColumn('uuid')
  id: string;

  // Nom optionnel pour une conversation de groupe.
  @Column({ length: 160, nullable: true })
  name?: string;

  // Indique s'il s'agit d'un groupe ou d'un chat direct.
  @Column({ name: 'is_group', default: false })
  isGroup: boolean;

  // Date de création de la room.
  @Column({ name: 'created_at', type: 'timestamp', default: () => 'NOW()' })
  createdAt: Date;

  // Liste des messages de la room.
  @OneToMany(() => Message, (message) => message.room)
  messages: Message[];

  // Liste des participants de la room.
  @OneToMany(() => RoomParticipant, (participant) => participant.room)
  participants: RoomParticipant[];
}
```

Entité RoomParticipant

```
// src/chat/entities/room-participant.entity.ts
import { Column, Entity, JoinColumn, ManyToOne, PrimaryColumn } from 'typeorm';
import { Room } from './room.entity';
import { User } from './user.entity';

@Entity('room_participants')
export class RoomParticipant {
  // Clé primaire composée : room_id.
  @PrimaryColumn({ name: 'room_id' })
  roomId: string;

  // Clé primaire composée : user_id.
  @PrimaryColumn({ name: 'user_id' })
```

```
  userId: string;

  // Date d'entrée dans la room.
  @Column({ name: 'joined_at', type: 'timestamp', default: () => 'NOW()' })
  joinedAt: Date;

  @ManyToOne(() => Room, (room) => room.participants, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'room_id' })
  room: Room;

  @ManyToOne(() => User, (user) => user.roomParticipations, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'user_id' })
  user: User;
}
```

21. Fonctionnement type Messenger

Pour ressembler davantage à Messenger, l'interface peut être découpée en trois zones : une colonne gauche avec la liste des conversations, une zone centrale avec l'historique, et une zone basse avec la saisie du message. Ce modèle convient très bien à React car chaque zone devient un composant distinct.

- ConversationList : affiche les rooms récentes, le dernier message et le nombre de non lus.
- MessageList : affiche les bulles de message selon leur type.
- MessageInput : gère texte, fichier, image, audio et indicateur de frappe.
- ChatPage : relie tous les composants, charge les données, écoute la socket et gère l'état global.

22. Gestion de l'état côté React

Au début, useState et useEffect suffisent. Quand l'application grandit, on peut introduire Context API, Zustand ou Redux. Pour un premier projet pédagogique, mieux vaut rester simple : l'historique des messages est stocké dans ChatPage, puis transmis aux composants enfants via les props.

23. Lecture des audios

Pour les notes vocales, le plus simple est d'envoyer un fichier audio enregistré côté navigateur ou choisi par l'utilisateur. Le backend l'upload via HTTP, puis le message audio est diffusé en WebSocket comme n'importe quel autre message.

Lecture audio

```
// Exemple d'affichage d'un audio déjà reçu.
<audio controls style={{ width: '100%' }}>
  <source
```

```
    src={`http://localhost:3000${message.fileUrl}`}
    type={message.mimeType}
  />
  Votre navigateur ne supporte pas la lecture audio.
</audio>
```

24. Bonnes pratiques de sécurité

- Valider toutes les données reçues avec des DTO et ValidationPipe.
- Vérifier l'appartenance d'un utilisateur à une room avant d'autoriser l'envoi ou la lecture des messages.
- Limiter les types MIME acceptés pour les uploads.
- Limiter la taille des fichiers.
- Hacher les mots de passe avec bcrypt.
- Ajouter une authentification JWT pour HTTP et WebSocket.
- Nettoyer ou échapper le contenu texte si l'affichage HTML devient plus riche.

25. Ordre de développement conseillé

13. Créer la base PostgreSQL.
14. Scaffolder le backend NestJS et configurer TypeORM.
15. Écrire les entités User, Room, RoomParticipant et Message.
16. Créer ChatService et le contrôleur GET /chat/rooms/:roomId/messages.
17. Créer le gateway et tester join_room puis send_message avec du texte simple.
18. Créer le frontend React et afficher l'historique d'une room.
19. Brancher la socket côté React et afficher new_message en direct.
20. Ajouter l'upload HTTP des images, fichiers et audios.
21. Améliorer l'interface : conversations, avatars, accusés de lecture, indicateur de frappe.
22. Ajouter ensuite l'authentification, les migrations et les tests.

26. Test manuel du projet

Une fois la base installée et les deux serveurs lancés, ouvre deux fenêtres de navigateur avec deux utilisateurs différents. Fais rejoindre la même room aux deux clients. Envoie ensuite un texte depuis la première fenêtre. Le message doit apparaître instantanément dans la seconde. Recommence avec une image, puis un fichier PDF, puis un fichier audio.

27. Pièges fréquents

- Le client React se connecte mais ne reçoit rien : vérifier le CORS et l'URL du socket.
- Les messages ne sont pas diffusés : vérifier que le client a bien rejoint la room avec join_room.
- Les fichiers ne s'ouvrent pas : vérifier ServeStaticModule et le chemin /uploads.

- Le message est sauvegardé mais ne s’affiche pas : vérifier les noms des événements new_message et send_message.
- Erreur PostgreSQL : vérifier les variables .env et le lancement du serveur Postgres.

28. Version minimale à retenir

Si tu veux retenir l’essentiel, pense à cette règle simple : le texte et les événements temps réel passent par WebSocket ; les gros contenus binaires passent d’abord par HTTP ; PostgreSQL conserve l’état durable ; React affiche immédiatement ce que NestJS diffuse à la room concernée.